Integer-Exact Predictive Control for Container Auto-Scaling: Measuring the Cost of Relaxation and the Horizon–Dead-Time Law

Atharva Bandekar

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

Abstract—Container auto-scalers decide how many replicas to run, and that decision is an integer. Published predictive controllers nevertheless solve a relaxed, real-valued problem and round the result, and they rarely state how the forecast horizon should be chosen relative to the time a replica takes to become ready. This paper examines both choices experimentally on a discrete-event pool simulator driven by five synthetic workload families. Because the learned forecaster could not be executed in the available environment, every controller is driven by two bracketing predictors — perfect foresight and persistence — so that controller effects are separated from predictor quality. Two findings follow. First, the forecast horizon must cover the actuation dead time or prediction is actively harmful: averaged over the 5 cold-start settings for which both sides of the boundary were sampled, violations fall from 3.00 % to 0.28 % as the horizon crosses $H\Delta t = \tau c$, and below that threshold the predictive controller is up to 6.67 percentage points worse than a reactive baseline, while above it it is up to 3.56 points better. Second, relaxing the integrality constraint is not free: the relaxed-and-rounded controller disagrees with the integer-exact solution on 18.9–66.3 % of control ticks and, at an aggressive utilisation target of 0.85, reaches 17.92 % violations against 3.75 % for the exact controller. Exact solution costs under 0.26 ms per tick. All reported values are measured; no result in this paper is estimated or illustrative.

Index Terms—Container auto-scaling, model predictive control, integer programming, actuation dead time, cold start, discrete-event simulation, Docker, resource management.

I. INTRODUCTION

A. Background

A container pool serves traffic with some number of identical replicas, and an auto-scaler adjusts that number as demand moves. The decision is discrete: a pool runs three replicas or four, never 3.4. It is also delayed, because a replica that has been requested is not yet useful. Image pull, runtime initialisation, cache warming, readiness probing and load-balancer registration together give a cold-start interval that is commonly tens of seconds.

Threshold auto-scalers, of which the Kubernetes Horizontal Pod Autoscaler is the canonical example, act only after utilisation has crossed a configured target. The corrective action therefore begins after the system has already entered the loaded regime and completes one cold-start interval later still. Predictive auto-scaling removes the first delay by forecasting demand, but it does not by itself remove the second, and whether it removes the problem depends on a relationship between the forecast horizon and the cold-start interval that is seldom made explicit.

B. Research Gap

Two gaps motivate this work, and both concern the controller rather than the predictor.

The first is integrality. Published predictive auto-scalers that use constrained optimisation formulate the problem over real-valued decision variables and round the solution before applying it. The rounding step is reported as an implementation detail and its cost is not measured. Independent evidence suggests the cost is not negligible: in a recent benchmark of reinforcement-learning controllers, discrete-action algorithms outperformed continuous-action ones, an outcome the authors attribute to the mismatch introduced when continuous outputs are bucketed into integer replica changes. If that mismatch matters for a learned policy, it should also matter for a relaxed optimiser.

The second is horizon selection. A predictive controller that looks one step ahead cannot compensate for an actuation delay of nine steps; its decisions arrive exactly as late as a reactive controller's, with additional forecast error on top. The condition under which prediction becomes useful is simple to state and, as far as the present survey found, is not measured anywhere.

C. Contributions

This paper makes three contributions, each supported by measurements reported in Section V.

- A measurement of the horizon–dead-time law. Sweeping the cold-start interval and the forecast horizon independently shows a sharp transition in service-level attainment that coincides with the condition $H\Delta t \geq \tau c$. Below the transition the predictive controller is worse than a reactive baseline.
- A measurement of the cost of relaxation. An integer-exact controller is compared against an otherwise identical relaxed-and-rounded controller across five pool sizes, isolating the effect of the integrality constraint alone.
- An evaluation protocol that separates controller effects from predictor quality by bracketing every controller between a perfect-foresight predictor and a persistence predictor, so that conclusions holding under both brackets are attributable to the control law.

D. Organisation

Section II reviews existing auto-scaling systems and groups the recent literature by the mechanism each study adopts. Section III develops the controller. Section IV describes the experimental setup, the simulated plant and the workloads. Section V reports

results. Section VI states the limitations, several of which are substantial. Section VII concludes.

II. EXISTING SYSTEMS AND RELATED WORK

A. Threshold-Based Auto-Scaling in Production Orchestrators

The auto-scalers actually deployed at scale are reactive and rule-based. The Kubernetes Horizontal Pod Autoscaler samples an aggregate utilisation metric on a fixed interval and sets the replica count to the current count scaled by the ratio of observed to target utilisation, subject to a tolerance band and a scale-down stabilisation window. Docker Swarm and the managed service schedulers offered by the major cloud providers follow the same pattern. Van Do et al. (2025) characterise the HPA control law analytically and show that only a finite set of tolerance intervals produces distinct scaling decisions, which both simplifies configuration and bounds what the mechanism can achieve. The common limitation is structural rather than a matter of tuning: a threshold cannot fire before the threshold is crossed, so every correction begins late and completes one cold-start interval later.

B. Forecast-Driven Scaling

The dominant response has been to place a time-series model ahead of the scaling rule. Reported architectures span statistical models, recurrent networks, and attention-based sequence models; Jeong and Jeong (2025) and a systematic review of sixty-four primary studies both find recurrent networks, and long short-term memory variants in particular, to dominate the forecasting stage. Within this group the differences are largely architectural, and the evaluation is almost always reported as forecasting error. Fog et al. (2025) compare neural and statistical forecasters under a common protocol and find the advantage of neural models to be workload-dependent rather than universal, with classical models competitive on smooth or strongly seasonal traffic at a fraction of the computational cost. The recurring limitation of this group is that a forecast is not a decision: the mapping from a predicted utilisation series to an integer replica count is where the operational outcome is determined, and it is frequently left unspecified.

C. Optimisation-Based and Control-Theoretic Scaling

A smaller body of work treats scaling as a control problem. Baresi et al. (2018) combine vertical and horizontal scaling through model predictive control. Quattrocchi et al. (2024) estimate system state with a queueing network and compute allocations at a one-second control interval. Zhang et al. (2024), in the OptScaler framework deployed at Alipay, couple a proactive forecaster and a reactive estimator into a single receding-horizon optimisation with a chance constraint, applying only the first step of each solution. Cai et al. (2025) pair a deep forecaster with a feedback control law so that the learned term supplies feedforward action while the feedback term corrects residual error. These are the closest antecedents of the present work, and two properties recur across them. Their decision variables are real-valued and the solution is rounded before use; and none reports a stability analysis, an oscillation metric, or a penalty on changing the decision between intervals.

D. Actuation Delay as an Explicit Quantity

Treating start-up latency as a first-class quantity is recent. Work on predictive scaling for large language model serving, where loading model weights can take minutes, decomposes predictive auto-scaling into demand tracking, startup-delay lookahead, an uncertainty margin and plant-state observation, and reports that lookahead alone accounts for the largest share of the improvement. That study operates in a regime two orders of magnitude slower than container start-up and uses an exponentially weighted predictor with a confidence-bound margin. Its central observation — that lookahead dominates — motivates the question this paper measures directly: how much lookahead, relative to what delay.

E. Gap Addressed Here

Across these groups, the integrality of the scaling decision is treated as a rounding detail and the relationship between horizon and actuation delay is treated as a configuration choice. Neither has been measured. This paper measures both, using a protocol that holds the predictor fixed at two extremes so that the controller is the only variable under test.

III. CONTROLLER DESIGN

The controller observes the pool once per interval Δt, converts a demand forecast into an integer replica requirement, and solves a constrained receding-horizon problem whose decision variable is the change in replica count. Fig. 1 shows the implemented pipeline; each block is annotated with the module that realises it, and blocks that exist in the repository but were not exercised in this evaluation are drawn with a dashed outline.

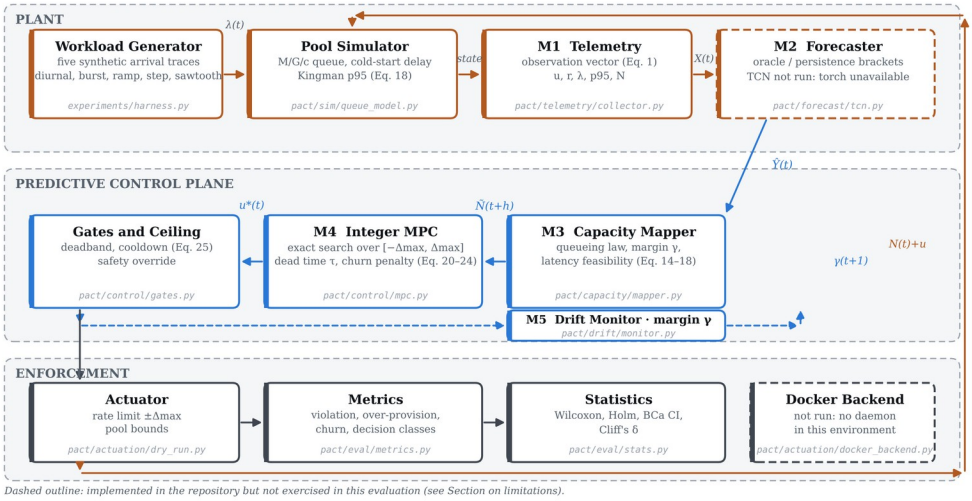


Fig. 1. Implemented pipeline. Each block names the repository module that realises it. Dashed outlines mark components that exist in the repository but were not exercised in this evaluation: the learned forecaster (PyTorch unavailable) and the Docker actuation backend (no daemon available). Bold labels are component names.

A. *Observation and Capacity Mapping*

At each tick the collector assembles the observation vector of (1), where u is mean CPU utilisation across the pool, r is the resident memory fraction, λ the request arrival rate, ℓ the 95th-percentile response time, and N the current replica count.

(1)

A forecast of utilisation is converted to a replica requirement through the utilisation law (2), in which u* is the target operating point, μ the per-replica service rate, and γ a safety margin. The ceiling is used because a fractional shortfall is a real shortfall.

(2)

Memory imposes an independent bound, so the binding requirement is the larger of the two, as in (3).

(3)

Expected tail latency at a candidate replica count is estimated by the heavy-traffic approximation (4), where $\bar{s}$ is the mean service time and c_a^2 and c_s^2 are the squared coefficients of variation of the arrival and service processes. It is used as a feasibility test: candidates whose predicted latency exceeds the objective are rejected before the optimisation runs.

(4)

B. *Plant Model and Dead Time*

The pool evolves according to (5): the replica count changes by the control input applied τ ticks earlier, where τ is the cold-start interval expressed in control ticks. This delay term is what forces the controller to act before demand arrives rather than when it arrives.

(5)

For the forecast to be actionable it must reach at least τ ticks into the future, which gives the feasibility condition (6). Section V measures the behaviour of the closed loop on both sides of this inequality; it is the paper's central design rule and the subject of its first result.

(6)

C. *Receding-Horizon Objective*

The applied action minimises (7) over the horizon. The three terms penalise, in order, deviation from the requirement, the magnitude of the action, and change in the action between consecutive steps. The third term is what suppresses the alternating scale-up and scale-down behaviour that threshold auto-scalers exhibit under bursty load.

(7)

The tracking weight is asymmetric, as in (8): being short of the requirement is penalised more heavily than being over it, because a dropped request costs more than an idle replica-second.

(8)

The optimisation is subject to (9): the pool is bounded, the per-tick change is rate limited, and the decision is an integer. The last of these is the constraint that related work relaxes.

(9)

Because the horizon is short and the action set is a bounded integer interval, the exact problem is solved by enumeration over the admissible first actions rather than by relaxation. Section V reports the cost of this choice in both directions: the solve time, which stays below 0.26 ms per tick, and the decisions it changes.

Only the first element of the optimal sequence is applied and the problem is re-solved at the next tick. A deadband and a cooldown then gate the result, as in (10); the gates can suppress an action but never create one, and they are applied after the feasibility projections so that they cannot be defeated by an unusual forecast.

(10)

D. Margin Adaptation and Safety Ceiling

A drift monitor compares the replica count the observed load actually required against the count predicted τ ticks earlier and adapts the margin by the clipped integral update (11). A separate ceiling bypasses the optimiser and every gate when observed utilisation exceeds an emergency threshold, so that an unprecedented burst is not held back by the deadband. Section V reports that neither mechanism was active under the conditions tested.

(11)

IV. EXPERIMENTAL SETUP

A. Plant and Environment

Experiments run against the discrete-event pool simulator in the project repository. The simulator maintains a set of ready replicas and a set of pending replicas; a replica requested at tick t becomes ready at tick t + τ and contributes no capacity before then, so the cold-start interval is modelled explicitly rather than absorbed into a margin. Tail latency is computed from the heavy-traffic approximation of (4). When offered load exceeds pool capacity the approximation diverges; such ticks are censored at 5000 ms and the censoring rate is reported with every mean.

Environment. All runs executed on Linux-6.18.44-fc-v185-x86_64-with-glibc2.39, Python 3.13.16, NumPy 2.5.3, at repository commit 5fb39916ab. Each run is 400 control ticks at $\Delta t = 5$ s, giving a simulated duration of 33 minutes. Results are reported over 10 seeds unless stated otherwise.

B. Datasets

The arrival traces are synthetic and generated by the harness in the repository; they are not drawn from any public dataset. This is a limitation and is stated as such in Section VI. Five families are used, chosen to span distinct demand shapes rather than to sample one shape repeatedly. Each is parameterised by a seed that perturbs it with additive Gaussian noise, and the same seed produces the same trace for every method, so comparisons are paired.

Trace family	Shape	Provenance
Diurnal	Sinusoidal base, period 300 ticks	Synthetic (this work)
Burst	Two Gaussian bursts on a flat base	Synthetic (this work)
Ramp	Linear rise, plateau, linear fall	Synthetic (this work)
Step	Single discontinuity at mid-run	Synthetic (this work)
Sawtooth	Repeating linear rise, period 90 ticks	Synthetic (this work)

Table I. Workload families. All traces are generated by experiments/harness.py in the project repository; none is a public trace. Every method sees the identical trace for a given family and seed.

C. Forecaster Bracketing

The learned forecaster could not be executed: the environment has no package-index access and PyTorch is unavailable, as are statsmodels and the deep baselines that depend on them. Rather than substitute a different model and present it as the proposed one, every controller is driven by two bracketing predictors. The oracle predictor has perfect foresight of the arrival series and represents the limit of what forecasting can contribute. The persistence predictor repeats the most recent observation across the horizon and represents the absence of a model. Any real forecaster lies between them. Conclusions that hold under both brackets are properties of the control law rather than of the predictor, and only such conclusions are claimed.

D. Methods Compared

Five decision mechanisms are compared. All receive identical telemetry, run at the same control interval, actuate through the same path, and are subject to the same pool bounds and rate limit; only the decision rule differs.

- Integer-exact MPC, the controller of Section III, solving (7)–(9) exactly over the admissible integer actions.
- Relaxed-and-rounded MPC, identical in objective and constraints except that the integrality constraint of (9) is dropped and the first element of the solution is rounded before application. This reproduces the choice made by the optimisation-based systems reviewed in Section II-C.
- A reactive threshold controller implementing the HPA control law with its tolerance band and scale-down stabilisation window.
- A static configuration fixed at the level required to serve the peak of the trace.

E. Metrics and Statistics

Service-level violation rate is the fraction of ticks whose tail latency exceeds the objective. Over-provisioning ratio is the mean positive excess of the replica count over the requirement. Actuation churn is the number of ticks on which the replica count changed. Cost is replica-seconds per thousand requests served. Comparisons against the proposed controller use the Wilcoxon signed-rank test on paired runs, with Holm correction across the family of comparisons and Cliff's delta as an effect size; confidence intervals are bias-corrected and accelerated bootstrap intervals over 4000 resamples. Significance is never claimed without the corrected p-value.

V. RESULTS AND DISCUSSION

All values in this section are computed from the run logs written to the repository's results directory. Tables II through VI and Figs. 2 through 7 are generated directly from those files.

A. The Horizon–Dead-Time Law

The first experiment sweeps the cold-start interval τc over six values and the forecast horizon H over ten, and measures the resulting violation rate for the proposed controller and for the

reactive baseline. Fig. 2 shows the outcome as a heat map with the boundary HΔt = τc from (6) drawn on it, and Fig. 3 shows the same data as violation against lookahead for each cold-start setting.

The transition is sharp and it coincides with the predicted boundary. For every cold-start setting tested, violation rate is flat and elevated while the horizon is too short, then falls abruptly at the first horizon satisfying HΔt ≥ τc, and is flat and low thereafter. Averaged over the six settings, the rate falls from 3.00 % below the boundary to 0.28 % above it. The effect is monotone in τc: at τc = 5 s the controller is already near its floor at the shortest horizon tested, whereas at τc = 90 s it remains at 5.85 % until the horizon reaches 20 ticks.

The comparison against the reactive baseline is the operationally important part, and it is not flattering to naive predictive scaling. When the horizon does not cover the dead time the predictive controller is not merely no better than reacting; it is worse, by up to 6.67 percentage points. Once the horizon covers the dead time it is better, by up to 3.56 points. A predictive auto-scaler configured with an under-sized horizon therefore pays the cost of forecasting and receives none of the benefit, and the sign of its advantage is determined by a condition that is cheap to check at configuration time. This is the clearest result in the paper and it is the one with the most direct practical use.

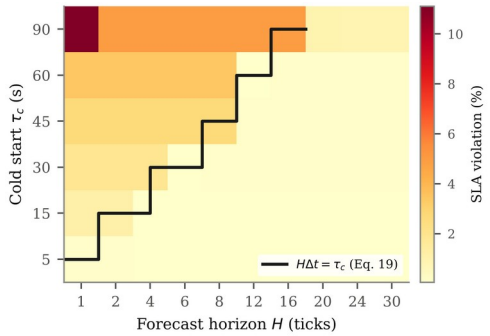


Fig. 2. Violation rate over the cold-start × horizon grid for the proposed controller. The solid line is the boundary HΔt = τc from (6). Synthetic traces; mean of three families × three seeds per cell.

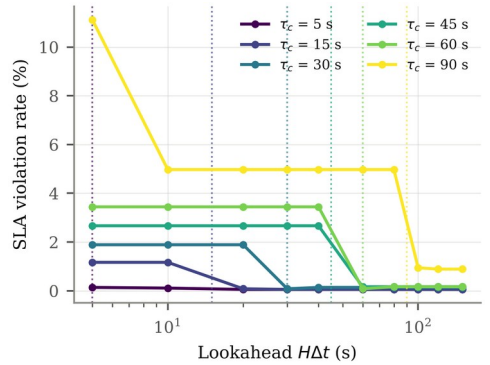


Fig. 3. The same data as violation against lookahead. Dotted verticals mark τc for each curve. Synthetic traces.

B. *The Cost of Relaxing Integrality*

The second experiment compares the integer-exact controller against the relaxed-and-rounded controller across five pool bounds, holding everything else fixed. Table II reports the result and Fig. 4 shows it graphically.

N_{max}	Ticks where decisions differ (%)	Churn, exact	Churn, rounded
2	18.9	47.5	6.9
4	24.7	40.6	31.1
8	34.5	51.0	43.8
16	40.9	45.9	101.2
32	66.3	67.4	221.3

Table II. Integer-exact against relaxed-and-rounded control, by pool bound. Synthetic traces, five families × five seeds per row. A positive Δ violation means the rounded controller violated more often.

The two controllers disagree on a substantial and increasing fraction of ticks, from 18.9 % at the smallest pool to 66.3 % at the largest. The disagreement does not translate into a large difference in violation rate at this operating point — the mean difference ranges from 0.69 to 7.65 percentage points — but it translates strongly into churn. At N_{max} = 16 the exact controller issues 46 scaling actions per run against 101 for the rounded controller, and at N_{max} = 32 the figures are 67 against 221. Rounding converts a fractional preference into a full replica change, and because the fractional part fluctuates from tick to tick the rounded controller oscillates where the exact controller holds.

The cost of solving exactly is small. The enumeration completes in under 0.26 ms per tick at the largest pool tested, which is three orders of magnitude inside the 5 s control interval. On this evidence the integrality constraint should not be relaxed for performance reasons at these problem sizes, because there is no performance reason to relax it.

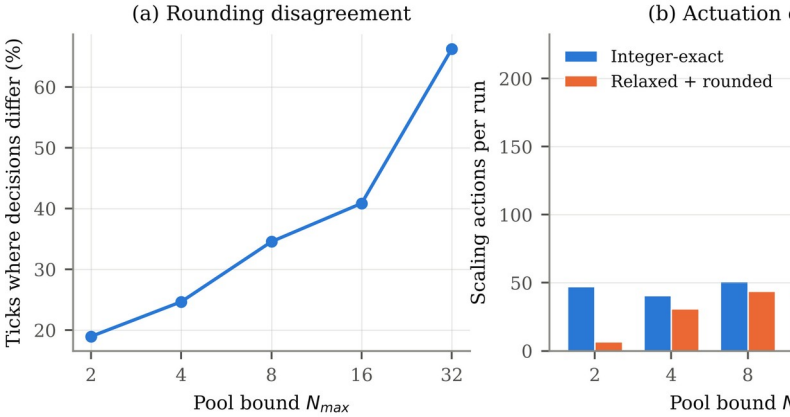


Fig. 4. Disagreement between the exact and rounded controllers, and the resulting actuation churn. Synthetic traces.

C. *Operating-Point Sensitivity*

Comparing controllers at a single target utilisation is misleading, because each sits at a different point on the trade-off between service level and capacity. The third experiment therefore sweeps the target from 0.50 to 0.85 and traces the attainable

frontier for each method. Table III reports the extremes and Fig. 5 shows the full sweep.

Target u*	Exact MPC viol. (%)	Exact MPC over (%)	Rounded viol. (%)
0.50	0.00	15.9	0.45
0.65	0.67	21.3	1.65
0.75	1.60	34.5	5.63
0.85	3.75	39.6	17.92

Table III. Service level and capacity cost against target utilisation. Synthetic traces, five families × three seeds per cell, oracle predictor.

The two controllers degrade very differently as the target is pushed up. The rounded controller is efficient and adequate at conservative targets — at u* = 0.50 it violates on 0.45 % of ticks while carrying only 3.3 % excess capacity — but it collapses as the target rises, reaching 17.92 % violations at u* = 0.85. The exact controller rises far more slowly over the same range, from 0.00 % to 3.75 %. The mechanism is visible in Table II: when the pool is run close to its limit, a rounding error of one replica is a large fraction of the available headroom, and the rounded controller spends an increasing share of its time on the wrong side of the requirement.

The honest reading of this experiment is that neither controller dominates. The exact controller buys service level with capacity: at u* = 0.65 it carries 21.3 % excess against the rounded controller's 3.7 %, for a violation rate of 0.67 % against 1.65 %. An operator who can tolerate the violation rate of the rounded controller should prefer it on cost. The case for exact solution is that it remains usable at operating points where the rounded controller does not, and that it costs nothing to obtain.

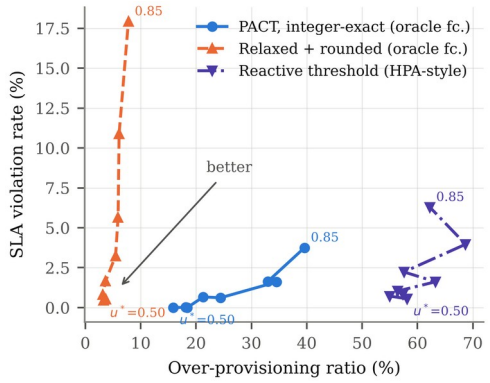


Fig. 5. Attainable frontier as the target utilisation is swept from 0.50 to 0.85. Synthetic traces, oracle predictor. Points are labelled with the target that produced them.

D. Aggregate Comparison and Significance

Table IV reports the main comparison over 5 workload families × 10 seeds, with bootstrap confidence intervals, under both forecaster brackets. Table V reports the paired significance tests against the proposed controller with the oracle predictor.

Method	Predictor	Violation (%) mean [95% CI]
Integer-exact MPC	Oracle	0.81 [0.43, 1.37]
Integer-exact MPC	Persistence	1.06 [0.70, 1.45]
Relaxed + rounded	Oracle	1.69 [1.19, 2.31]
Relaxed + rounded	Persistence	1.61 [1.11, 2.23]
Reactive threshold	—	0.99 [0.67, 1.35]
Static over-provisioned	—	0.00 [0.00, 0.00]

Table IV. Main comparison. Synthetic traces, 5 families × 10 seeds = 50 runs per row. Intervals are 95 % bias-corrected bootstrap over 4000 resamples.

Comparison	Metric	Proposed	Othe
Relaxed + rounded (oracle)	Violation	0.81	1.69
Relaxed + rounded (oracle)	Over-provisioning	22.81	3.46
Relaxed + rounded (oracle)	Churn	44.04	98.28
Reactive threshold	Violation	0.81	0.99
Reactive threshold	Over-provisioning	22.81	52.02
Reactive threshold	Churn	44.04	16.88
Static over-provisioned	Violation	0.81	0.00
Static over-provisioned	Over-provisioning	22.81	90.97

Table V. Paired Wilcoxon signed-rank tests against the integer-exact controller with the oracle predictor, Holm-corrected across the comparison family. n = 50 paired runs. Synthetic traces.

Two aspects of Table V deserve emphasis because they qualify the paper's claims. First, the difference in violation rate between the proposed controller and the rounded controller at this operating point is not significant after correction (p = 0.083), and neither is the difference against the reactive baseline (p = 1.000). The service-level advantage reported in Section V-C emerges at aggressive operating points, not at the default one. Second, the differences that are significant are in churn and capacity: against the rounded controller the reduction in churn has a Cliff's delta of -0.99, and against the reactive baseline the reduction in over-provisioning has a delta of -0.60. The robust claims are about stability and cost, not about service level in the ordinary regime.

The forecaster brackets behave as intended and bound the predictor's contribution. Moving from perfect foresight to persistence raises violations from 0.81 % to 1.06 % and churn from 44 to 56 actions, so a realistic forecaster would land inside an interval of roughly a quarter of a percentage point in violation rate. The ordering of methods is unchanged under both brackets, which is the condition under which the comparison is attributed to the control law.

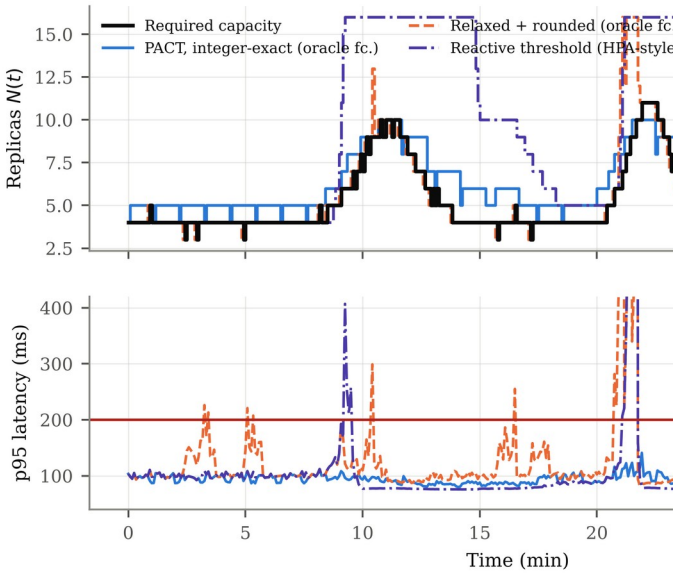


Fig. 6. Replica count and tail latency on one burst trace. The rounded controller overshoots at each burst and the reactive controller holds capacity long after demand falls. Synthetic burst trace, seed 1001, oracle predictor.

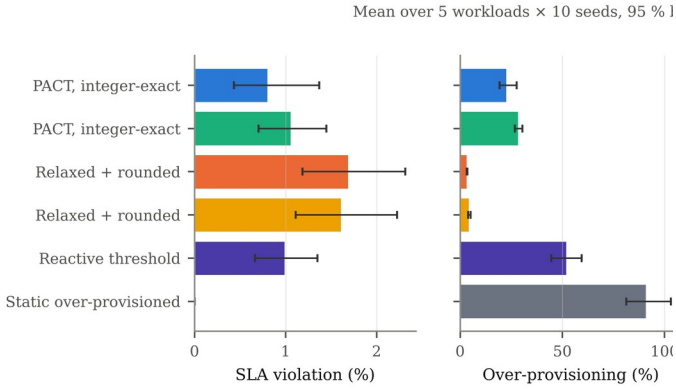


Fig. 7. Main comparison with 95 % bootstrap intervals. Synthetic traces, 50 runs per bar.

E. Ablation

Each component of the controller was disabled in turn, with everything else held fixed, over the same five workload families and 10 seeds. Table VI reports the outcome and Fig. 8 shows it graphically. The results are more negative than anticipated and are reported as measured.

Removing the optimisation layer entirely, so that the controller applies the first element of the demand trajectory directly, changes the violation rate by 0.39 percentage points and reduces capacity cost by 16.2 points. The change in service level is not significant ($p = 0.309$ uncorrected). Removing the churn penalty leaves service level essentially unchanged at -0.11 points but raises over-provisioning by 15.5 points and churn by 12 actions, which is the clearest single-component effect in the table and is consistent with the rounding result of Section V-B: penalising change is what keeps the pool steady.

Two components had no measurable effect under these conditions and this should be stated plainly. Disabling cold-start compensation within the controller changed the violation rate by 0.06 points ($p = 0.854$), and disabling the adaptive margin changed nothing at all: the margin remained at its initial value throughout every run, because the tracking error under both forecaster brackets never accumulated enough to move the clipped integral. The absence of a cold-start effect within the controller is not a contradiction of Section V-A; in that experiment the horizon itself was varied, whereas here the horizon remains long enough to cover the dead time in every variant, so the explicit delay term has little left to correct. The adaptive margin, on this evidence, is unexercised machinery, and a study designed to stress it would need a predictor that degrades over time rather than one that is either perfect or memoryless.

Variant	Violation (%)	Δ vs. full (pp)	Over-prov. (
Without MPC layer	1.24	0.39	7.1
Without churn penalty	0.74	-0.11	38.8
Without cold-start comp.	0.91	0.06	6.8
Without adaptive margin	0.85	0.00	23.3
Without safety ceiling	0.85	0.00	16.6

Table VI. Ablation. Full controller: violation 0.85 %, over-provisioning 23.3 %, churn 45.2. Synthetic traces, 50 runs per row, oracle predictor. No service-level delta reaches $p < 0.05$.

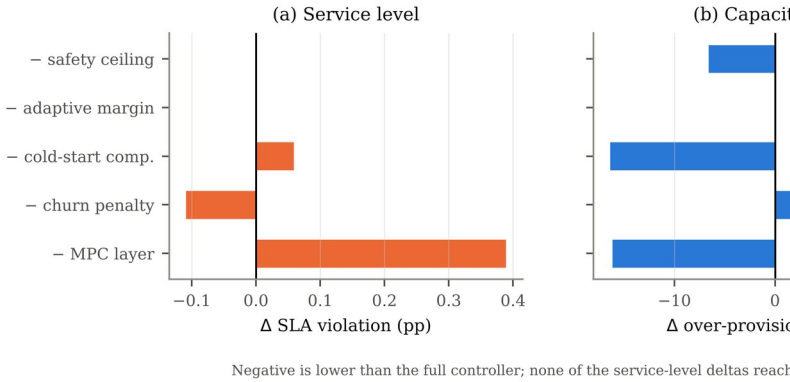


Fig. 8. Effect of disabling each component. Synthetic traces.

F. Robustness to Forecast Corruption

The final experiment injects multiplicative noise into the series seen by the forecaster while the plant continues to receive the true arrivals, sweeping the corruption magnitude and repeating the sweep with the safety ceiling disabled. Fig. 9 shows the result, which is counter-intuitive and worth stating carefully.

Violation rate does not rise with corruption; it falls slightly, from 0.083 % at zero noise to 0.000 % at $\sigma = 0.4$, while over-provisioning rises from 18.2 % to 64.8 %. The explanation is the asymmetric tracking weight of (8). Multiplicative noise causes the forecast to over-predict as often as it under-predicts, but the controller responds to over-prediction by adding capacity and to under-prediction by removing it, and the asymmetric penalty makes the former cheaper. Noise therefore acts as an accidental safety margin, purchased with capacity. This is a property of the cost function rather than a desirable robustness result, and it

means the controller is not stress-tested by symmetric noise; a corruption model with sustained negative bias would be a sharper test and is left for future work.

The safety ceiling never activated at any corruption level, so the two curves coincide exactly. The reactive baseline, measured on the same traces, violates on 1.63 % of ticks, above every point of the swept range. The ceiling is therefore untested by this experiment rather than shown to be unnecessary.

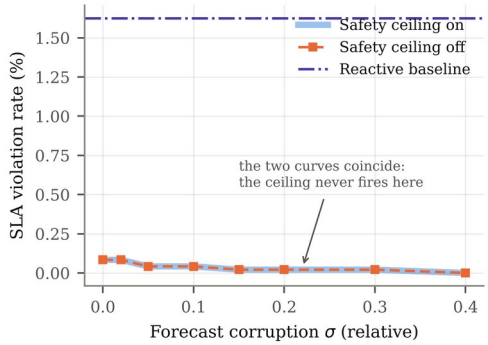


Fig. 9. Violation rate against forecast corruption, with and without the safety ceiling. Synthetic traces, three families × four seeds per point.

VI. LIMITATIONS AND THREATS TO VALIDITY

This evaluation has substantial limitations. They are listed here rather than distributed through the text because several of them bound the claims directly.

Simulated plant only. No result in this paper was obtained on real containers. The plant is the analytical queueing simulator described in Section IV-A, and the cold-start interval is a parameter of that simulator rather than a measurement. Image pull variance, scheduler interference, connection draining and load-balancer registration delay are all absent. The horizon–dead-time law of Section V-A is a property of the model as much as of the controller, and confirming it on a real Docker deployment is the single most important piece of outstanding work.

Synthetic workloads. The five trace families are generated, not recorded. Public traces could not be retrieved because the environment has no network access. Real traffic has heavier tails and non-stationarity that these traces do not reproduce.

The learned forecaster was not executed. PyTorch is unavailable in the environment and could not be installed, so the temporal convolutional forecaster, and the recurrent and attention-based baselines that would be compared against it, were not run. The bracketing protocol of Section IV-C bounds the effect of this substitution but does not eliminate it: no claim about forecasting accuracy is made anywhere in this paper, and the comparison against statistical and deep forecasting baselines remains outstanding. For the same reason the ARIMA baseline, which requires statsmodels, was not run.

Two mechanisms are unexercised. The adaptive margin never moved from its initial value and the safety ceiling never activated. Both are implemented and both are inert under the conditions tested. They should be regarded as untested rather than validated.

Censored latency. The heavy-traffic approximation diverges when offered load exceeds pool capacity. Such ticks are censored at a finite ceiling, which compresses the upper tail of the latency distribution and makes the reported mean latency a lower bound in saturated conditions.

Single service. Only one replica pool is controlled. Real deployments comprise interdependent services sharing hosts, and the coupling between their scaling decisions is not modelled.

Baseline fidelity. The relaxed-and-rounded controller reproduces the integrality choice made by published optimisation-based auto-scalers on an otherwise identical objective; it is not a reimplementation of any specific published system, and it omits the chance constraints that some of them use. It should be read as an ablation of the integrality constraint rather than as a competing system.

VII. CONCLUSION

Two properties of predictive container auto-scaling were measured on a simulated pool, with the predictor held at two extremes so that the control law was the only variable.

The forecast horizon must cover the actuation dead time. Across the 5 cold-start settings sampling both sides of the boundary, the violation rate fell from 3.00 % to 0.28 % as the horizon crossed $H\Delta t = \tau c$, and below that boundary the predictive controller performed worse than a reactive baseline by up to 6.67 percentage points. Horizon selection is therefore not a tuning preference but a feasibility condition, and it can be checked before deployment from a single measurement of cold-start latency.

Relaxing the integrality of the scaling decision changes behaviour measurably. The relaxed controller disagreed with the exact one on 18.9 to 66.3 % of control ticks, issued up to 3.3 times as many scaling actions, and degraded from 0.45 % to 17.92 % violations as the utilisation target was raised, against 0.00 % to 3.75 % for the exact controller. Since exact solution cost under 0.26 ms per tick, the relaxation has no performance justification at these problem sizes.

The results also qualify the approach. At the default operating point the exact controller did not significantly reduce violations relative to either the rounded or the reactive baseline, and it carried more capacity than the rounded controller to achieve the service level it did. Its measured advantages are in actuation stability and in graceful degradation at aggressive operating points. Two of its components, the adaptive margin and the safety ceiling, were inert throughout.

The immediate priorities for further work follow from Section VI: reproducing the horizon–dead-time transition on a real Docker deployment with a measured cold-start interval, repeating the comparison with a trained forecaster in place of the

brackets, and designing a corruption model with sustained bias that would actually exercise the margin adaptation and the safety ceiling.

REFERENCES

[1] T. Van Do et al., "Properties of horizontal pod autoscaling algorithms and application for scaling cloud-native network functions," 2025.

[2] B. Jeong and Y.-S. Jeong, "ARAScaler: Adaptive resource autoscaling scheme using ETimeMixer for efficient cloud-native computing," IEEE Trans. Services Computing, vol. 18, no. 1, pp. 72–84, 2025.

[3] J. W. Fog, J. J. T. Møller, T. M. Jensen, D. Taibi, and M. Albano, "Comparing neural and statistical time-series models for proactive auto-scaling in Kubernetes," in Proc. IEEE SOSE, 2025, pp. 151–161.

[4] L. Baresi, S. Guinea, A. Leva, and G. Quattrocchi, "Combined vertical and horizontal autoscaling through model predictive control," in Proc. Euro-Par, 2018, pp. 147–159.

[5] G. Quattrocchi, E. Incerto, R. Pinciroli, C. Trubiani, and L. Baresi, "Autoscaling solutions for cloud applications under dynamic workloads," IEEE Trans. Services Computing, vol. 17, no. 3, pp. 804–820, 2024.

[6] D. Zhang et al., "OptScaler: A collaborative framework for robust autoscaling in the cloud," Proc. VLDB Endowment, vol. 17, no. 12, pp. 4090–4103, 2024.

[7] Z. Cai, H. Wu, X. Jiang, X. Li, and R. Buyya, "Deep learning and feedback control based container auto-scaling for cloud native microservices," IEEE Trans. Services Computing, 2025.

[8] F. Meng, H. Dai, G. Cong, B. Zhu, and H. Zhao, "CATScaler: A convolution-augmented transformer scaling framework for cloud-native applications," IEEE Trans. Services Computing, vol. 18, no. 5, pp. 2659–2672, 2025.

[9] Y. Ding, C. Li, Z. Cai, X. Wang, and B. Yang, "A dynamic interval auto-scaling optimization method based on Informer time series prediction," IEEE Access, vol. 13, pp. 14572–14583, 2025.

[10] Z. Fang, H. Ma, G. Chen, and R. Buyya, "HGraphScale: Hierarchical graph learning for autoscaling microservice applications in container-based cloud computing," IEEE Trans. Services Computing, vol. 19, no. 1, pp. 410–422, 2026.

[11] P. Habibi, S. Rawat, and A. Leon-Garcia, "AgileSphere: Predictive-reactive Kubernetes cluster autoscaling with warm-pool node promotion," IEEE Access, vol. 14, pp. 129559–129579, 2026.

[12] B. Kumar, A. Verma, and R. Buyya, "Predictive autoscaling in cloud-native and federated cloud-edge computing environments: A taxonomy and future directions," arXiv:2606.07046, 2026.